Component Specification and Interface Contracts

Civic Operations · Group 23 · contract version 1.0.0 (LOCKED) · 18 September 2026

This is a suggested solution, not a fixed prescription. The architecture, techniques and steps here are designed so the 14 components work best together. Each owner may choose a different technique if it serves the component better; what must hold is the locked input/output contract and the safety rules, so the parts keep integrating. Changes to either go through a Contract Change Request.

What this document is. For each component: purpose, locked inputs and outputs, how it works inside, why it is better than the baseline, its safe default, how it is evaluated, and the business outcome. The field tables are generated from `packages/contracts`, so they always match the code. **Build your component however you like; do not change its inputs or outputs without a Contract Change Request** (Team Build Guide, Section 4).

Shared building blocks

Every output carries `meta` (who produced it, version, status OK / DEGRADED / FAILED, latency) and a list of **Reason** objects (code, message, weight, evidence). The admin drill-down shows these reasons verbatim; this is how every decision is explained.

Reason

Field	Type	Required	Meaning
code	str	yes	e.g. NEAR_HOSPITAL
message	str	yes	e.g. Request is 140 m from Kings County Hospital
weight	float?	no	Signed contribution to the score, if any
evidence	dict[str, Any]	no	

RequestFacts: the read-only view of a request that per-request components receive

Field	Type	Required	Meaning
request_id	str	yes	
created_at	datetime	yes	
borough	Borough	yes	
request_type_code	str	yes	
complaint_type	str	yes	
descriptor	str?	no	
complaint_family	str	yes	
location_type	str?	no	
incident_address	str?	no	
incident_zip	str?	no	
address_type	str?	no	
bbl	str?	no	
latitude	float?	no	
longitude	float?	no	
channel	str	yes	
citizen_id	str?	no	
free_text	str?	no	

Canonical request record and citizen view

IntakeRequest: what the citizen portal submits

Field	Type	Required	Meaning
citizen_id	str	yes	e.g. CIT-000042
channel	Channel	yes	
borough	Borough	yes	
request_type_code	str	yes	Code from data/reference/request_catalog.csv
incident_address	str?	no	
latitude	float?	no	
longitude	float?	no	
location_type	str?	no	e.g. Street
free_text	str?	no	Optional note; always screened by C02 and C03
bot_session_id	str?	no	Set when the bot filled the form
client_submission_id	str	yes	Idempotency key generated by the browser; a retry with the same key returns the same request

CitizenStatusView: what the citizen tracker shows (no internal scores, ever)

Field	Type	Required	Meaning
request_id	str	yes	
stage	CitizenStage	yes	
status	RequestStatus	yes	
headline	str	yes	e.g. A drainage crew is scheduled
agency_name	str?	no	
crew_type	str?	no	e.g. Sewer & drainage crew
eta_from	datetime?	no	
eta_to	datetime?	no	
linked_reports	int	no	'You and 3 others reported this'
updates	list[StatusUpdate]	no	

C01 · NLP Intake Bot

Owner	Technique group	Acceptance tests
Sankalp	Cognitive Systems	services/api/tests/test_api.py::test_bot_parse_and_rate_limit

Purpose

Lets a citizen who cannot or does not want to fill a form describe the problem in their own words, typed or spoken (the browser's speech-to-text). It fills the same structured request as the form and asks for whatever is missing.

Inputs (locked)

IntakeParseInput

Field	Type	Required	Meaning
session_id	str	yes	
utterance	str	yes	
is_voice	bool	no	
history	list[BotTurn]	no	
borough_hint	Borough?	no	
gps	GeoPoint?	no	

Outputs (locked)

IntakeParseResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
intent	IntentCandidate	yes	
alternatives	list[IntentCandidate]	no	
slots	Slots	yes	
missing_slots	list[str]	no	e.g. ['incident_address']
next_question	str?	no	e.g. Which street is the drain on?
ready_to_submit	bool	no	
reasons	list[Reason]	no	

How it works inside

1. Every turn is first screened by C03 (and C02) so an emergency is caught on the first sentence.
2. An intent classifier maps the text to one of the 34 catalog request types, or EMERGENCY, SMALLTALK or OTHER.
3. A slot extractor (regex baseline vs token classifier) pulls out borough, address, cross street, time and hazard words.
4. Dialogue policy: low confidence leads to a confirmation question; a missing slot leads to a targeted question. ready_to_submit only when the intent is confident and the location is known. The bot never files by itself; the citizen confirms.

Why it works and why it is better

A keyword bot breaks on real speech (72% gold macro-F1). A light learned model reaches 88 to 93% while staying under 20 ms on a free CPU, so the bot is intelligent but not resource heavy. Choosing by accuracy AND latency AND size, on a hand-written test set, is what makes the choice defensible.

Safe default (used automatically if it fails)

ready_to_submit = False and the citizen is offered the form. The bot never files anything on its own.

Evaluation (technical)

- Intent accuracy and macro-F1 on the hand-written gold set (not the templates)
- Slot span F1 (precision and recall)
- Latency ms per utterance and model size (deployability)
- Turns to completion in scripted dialogues

Business outcome

- Accessibility: citizens who cannot type can still report
- Fewer incomplete or miscategorised requests
- Share of bot sessions that end in a valid submission

C02 · Profanity Filter

Owner	Technique group	Acceptance tests
Khalid	Cognitive Systems / Decision Automation	services/api/tests/acceptance/test_c02_profanity.py

Purpose

Screens every piece of free text (form notes, bot turns, crew notes) for abusive language before storage, so staff never read abuse and the record stays professional, without ever blocking a real problem report.

Inputs (locked)

TextScreenInput

Field	Type	Required	Meaning
text	str	yes	
source_field	str	yes	e.g. free_text
citizen_id	str?	no	

Outputs (locked)

ProfanityResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
is_profane	bool	yes	
severity	ProfanitySeverity	yes	
score	float	yes	
masked_text	str	yes	
matched_count	int	yes	How many terms matched; the terms themselves are not returned
action	ProfanityAction	yes	
abuse_flag	bool	no	Directed abuse at staff; C07 reads this
reasons	list[Reason]	no	

How it works inside

1. Normalise (case, leetspeak such as f*ck and sh1t, repeated letters).
2. Lexicon match with word boundaries (data/reference/profanity_lexicon.csv, MILD/SEVERE).
3. A classifier for context and misspellings, benchmarked against the lexicon.
4. Directed-abuse detection ('you useless idiots') sets abuse_flag, which C07 can read.
5. Action: MILD leads to MASK and the request is filed normally; SEVERE directed abuse leads to ASK_REPHRASE.

Why it works and why it is better

Masking instead of blocking keeps the complaint (angry citizens often have real problems) while protecting staff. A classifier catches obfuscated spellings a list misses; the lexicon keeps it explainable.

Safe default (used automatically if it fails)

ALLOW, with the text unchanged (the pipeline continues).

Evaluation (technical)

- Precision, recall, F1 on the gold set profanity column
- False-positive rate on clean text (Scunthorpe cases such as 'Hell's Kitchen', 'cockroach')
- Latency

Business outcome

- Abusive messages reaching staff per month
- Complaints wrongly masked (harm measure)

C03 · Emergency Safeguard (911 redirect)

Owner	Technique group	Acceptance tests
Sankalp	Decision Automation / Cognitive Systems	services/api/tests/test_api.py::test_emergency_stops_and_stores_nothing

Purpose

311 is for non-emergencies. Any text that suggests a medical emergency, fire, gas leak, crime in progress, violence or a life-safety hazard stops the flow before anything is stored and shows a full-screen 'Call 911 now'.

Inputs (locked)

TextScreenInput

Field	Type	Required	Meaning
text	str	yes	
source_field	str	yes	e.g. free_text
citizen_id	str?	no	

Outputs (locked)

EmergencyResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
is_emergency	bool	yes	
category	EmergencyCategory	yes	
confidence	float	yes	
action	EmergencyAction	yes	
matched_signals	list[str]	no	e.g. ['not breathing']
reasons	list[Reason]	no	

How it works inside

1. Weighted regex rules per category (data/reference/emergency_lexicon.csv) run in-process, always.
2. Negation handling ('nobody is hurt') lowers confidence and leads to a confirm question instead of STOP.
3. Look-alikes are excluded ('fire hydrant' and 'fire escape' are not fire).
4. A classifier adds recall: rules OR model; the model can add a hit but never remove a rule hit.

Why it works and why it is better

Missing an emergency is the worst possible error, so the design is recall-first and does not depend on any model being available.

Safe default (used automatically if it fails)

CONFIRM_WITH_CITIZEN: the 911 advisory is always shown (fail closed).

Evaluation (technical)

- Recall on gold emergencies (target 1.0 before demo)
- Precision, and false alarms on look-alike lines (hydrant, fire escape)

Business outcome

- Emergencies correctly redirected
- Non-emergencies wrongly redirected (friction)

C04 · Agency & Skill Routing (new)

Owner	Technique group	Acceptance tests
Mohammad	Decision Automation + Knowledge Discovery	services/api/tests/acceptance/test_c04_agency_routing.py

Purpose

Decides which city agency owns the request and which crew skill it needs, dynamically, so each request goes to the right agency and only a qualified crew is ever assigned.

Inputs (locked)

RequestFacts

Field	Type	Required	Meaning
request_id	str	yes	
created_at	datetime	yes	
borough	Borough	yes	
request_type_code	str	yes	
complaint_type	str	yes	
descriptor	str?	no	
complaint_family	str	yes	
location_type	str?	no	
incident_address	str?	no	
incident_zip	str?	no	
address_type	str?	no	
bbl	str?	no	
latitude	float?	no	
longitude	float?	no	
channel	str	yes	
citizen_id	str?	no	
free_text	str?	no	

Outputs (locked)

AgencyRoutingResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
agency	AgencyCode	yes	
agency_name	str	yes	
required_skill	CrewSkill	yes	
secondary_agency	AgencyCode?	no	
routing_confidence	float	yes	
method	str	yes	
dispatchable	bool	yes	False means REFERRED, never sent to a crew
needs_human_review	bool	no	
reasons	list[Reason]	no	

How it works inside

1. Rule table: request_catalog.csv + NYC311_agency_matrix.md (including multi-agency families such as a blocked street by cause).
2. Classifier on real rows: predicts agency from complaint type, descriptor, location type and free text; trained on the real agency column.
3. Real referrals (outcome_class = REFERRED, e.g. DOT asking DEP) teach where the first agency is wrong.
4. Rules + model agreement leads to high confidence; disagreement leads to secondary_agency and human review.

Why it works and why it is better

Rules alone are rigid for text-only (bot) requests and ambiguous cases; a model alone is not auditable. The hybrid is explainable and learns from real mis-routings.

Safe default (used automatically if it fails)

The catalog's default agency and skill, method DEFAULT, needs_human_review = True.

Evaluation (technical)

- Accuracy and macro-F1 (time-based test split)
- Agency confusion matrix
- Recall on real referral cases
- Rules vs model vs hybrid

Business outcome

- Mis-routed requests (re-assignments) per month
- Time lost to referrals between agencies

C05 · Duplicate & Cross-Channel Detection

Owner	Technique group	Acceptance tests
Nabeel	Knowledge Discovery + Decision Automation + Cognitive Systems	services/api/tests/acceptance/test_c05_duplicate.py

Purpose

Groups reports of the same physical incident (often from different people and channels within minutes) so one crew goes once, every reporter is informed, and corroboration becomes evidence of how real the problem is.

Inputs (locked)

RequestFacts

Field	Type	Required	Meaning
request_id	str	yes	
created_at	datetime	yes	
borough	Borough	yes	
request_type_code	str	yes	
complaint_type	str	yes	
descriptor	str?	no	
complaint_family	str	yes	
location_type	str?	no	
incident_address	str?	no	
incident_zip	str?	no	
address_type	str?	no	
bbl	str?	no	
latitude	float?	no	
longitude	float?	no	
channel	str	yes	
citizen_id	str?	no	
free_text	str?	no	

Outputs (locked)

DuplicateResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
relation	DuplicateRelation	yes	
duplicate_group_id	str	yes	
is_primary	bool	yes	
duplicate_of	str?	no	
recurrence_of	str?	no	
match_confidence	float	yes	
corroboration_count	int	no	Distinct reporters in the group
distinct_channel_count	int	no	
auto_close_recommended	bool	no	
match_evidence	list[Reason]	no	
reasons	list[Reason]	no	

How it works inside

1. Exact key: bbl + complaint type within a time window.
2. Rule baseline: radius + time window (the benchmark).
3. DBSCAN spatio-temporal clustering vs HDBSCAN, plus fuzzy address matching; address_type decides how much to trust coordinates.
4. NetworkX connected components assemble pairs into groups and give the evidence trail.
5. Only OPEN requests absorb duplicates; the same place and type after closure is a RECURRENCE (goes to C07 escalation).

Why it works and why it is better

Counting the same incident many times inflates the queue and sends several crews; ignoring it hides corroboration. Density clustering adapts to dense vs quiet areas, where a fixed radius fails.

Safe default (used automatically if it fails)

relation NEW, is_primary True. It never merges on error.

Evaluation (technical)

- Precision, recall, F1 on about 300 hand-labelled real pairs
- False merge rate (the expensive error)
- Adjusted Rand Index, silhouette

Business outcome

- Crew trips avoided
- Reduction in the operator's working queue
- Citizens informed per resolved job
- Operator override rate

C06 · Fake & Unfounded Request Detection

Owner	Technique group	Acceptance tests
Nabeel	Decision Automation + Knowledge Discovery	services/api/tests/acceptance/test_c06_fake.py

Purpose

Scores how likely a request describes a problem that is not there, so wasted trips fall, without ever refusing a citizen.

Inputs (locked)

RequestFacts

Field	Type	Required	Meaning
request_id	str	yes	
created_at	datetime	yes	
borough	Borough	yes	
request_type_code	str	yes	
complaint_type	str	yes	
descriptor	str?	no	
complaint_family	str	yes	
location_type	str?	no	
incident_address	str?	no	
incident_zip	str?	no	
address_type	str?	no	
bbl	str?	no	
latitude	float?	no	
longitude	float?	no	
channel	str	yes	
citizen_id	str?	no	
free_text	str?	no	

DuplicateResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
relation	DuplicateRelation	yes	
duplicate_group_id	str	yes	
is_primary	bool	yes	
duplicate_of	str?	no	
recurrence_of	str?	no	
match_confidence	float	yes	
corroboration_count	int	no	Distinct reporters in the group
distinct_channel_count	int	no	
auto_close_recommended	bool	no	
match_evidence	list[Reason]	no	
reasons	list[Reason]	no	

CredibilityResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
credibility_score	float	yes	
observation_count	int	yes	
credibility_band	str	yes	
escalation_flag	bool	no	
escalation_level	EscalationLevel	no	
recurrence_count	int	no	Closures at this place and type in the window
evidence_trail	list[str]	no	request_ids proving the pattern
reasons	list[Reason]	no	

Outputs (locked)

FakeRiskResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
unfounded_probability	float?	no	
risk_band	RiskBand?	no	
recommended_action	FakeAction	no	
reporter_burst_flag	bool	no	
contributing_factors	list[Reason]	no	
reasons	list[Reason]	no	

How it works inside

1. Label from real closing notes (outcome_class); inconclusive rows are excluded; resolution_description is never a feature.
2. Rule guardrail on reporter history and bursts; logistic regression (readable coefficients) vs gradient boosted trees.
3. Features: type base rates, location history, time, corroboration (C05), credibility (C07), burst behaviour.
4. Calibrated probability, then a risk band, then PROCEED / LOWER_PRIORITY / HOLD_FOR_REVIEW (never reject).

Why it works and why it is better

A proxy label from real outcomes beats invented labels; calibration matters because the score changes a citizen's priority.

Safe default (used automatically if it fails)

unfounded_probability None, action PROCEED: a failure never deprioritises anyone.

Evaluation (technical)

- AUC ROC, F1 at the operating point, Brier score
- Precision@K
- False-positive rate on first-time reporters
- Recall on high-urgency types

Business outcome

- Wasted crew trips avoided per month
- Genuine requests wrongly deprioritised (always reported alongside)

C07 · Escalation, Credibility & Outcome Verification

Owner	Technique group	Acceptance tests
Sankalp	Decision Automation + Knowledge Discovery	services/api/tests/test_api.py::test_closure_loop_manual_assign

Purpose

Escalates problems that keep coming back to a supervisor or the central authority, keeps a fair and recoverable trust score per citizen, and turns each crew outcome into verified evidence for everyone who reported the incident.

Inputs (locked)

RequestFacts

Field	Type	Required	Meaning
request_id	str	yes	
created_at	datetime	yes	
borough	Borough	yes	
request_type_code	str	yes	
complaint_type	str	yes	
descriptor	str?	no	
complaint_family	str	yes	
location_type	str?	no	
incident_address	str?	no	
incident_zip	str?	no	
address_type	str?	no	
bbl	str?	no	
latitude	float?	no	
longitude	float?	no	
channel	str	yes	
citizen_id	str?	no	
free_text	str?	no	

CrewOutcomeReport

Field	Type	Required	Meaning
assignment_id	str	yes	
request_id	str	yes	
crew_id	str	yes	
reported_at	datetime	yes	
outcome	CrewOutcomeCode	yes	
note	str?	no	Screened by C02 before storage
on_site_minutes	float?	no	
photo_path	str?	no	Supabase Storage path, EXIF stripped

Outputs (locked)

CredibilityResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency

credibility_score	float	yes	
observation_count	int	yes	
credibility_band	str	yes	
escalation_flag	bool	no	
escalation_level	EscalationLevel	no	
recurrence_count	int	no	Closures at this place and type in the window
evidence_trail	list[str]	no	request_ids proving the pattern
reasons	list[Reason]	no	

OutcomeVerificationResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
request_id	str	yes	
verified_outcome	VerifiedOutcome	yes	
confidence	float	yes	
citizens_updated	list[str]	no	
credibility_deltas	dict[str, float]	no	
reasons	list[Reason]	no	

How it works inside

1. Recurrence rule: repeated resolved requests of the same type within 150 m / 180 days lead to SUPERVISOR (2) or CENTRAL_AUTHORITY (4), with the evidence trail.
2. Beta(2,2) trust model: GENUINE adds to alpha, UNFOUNDED adds to beta, INCONCLUSIVE is ignored; score = 100 * alpha / (alpha + beta); 180-day half-life decay toward neutral.
3. A new citizen starts at 50 and is never judged on one report (the prior).
4. Closure: crew outcome to verified outcome (SUBSIDED counts as GENUINE); credibility updated for EVERY reporter in the duplicate group.

Why it works and why it is better

A Bayesian score handles small samples fairly and is recoverable, unlike a point counter; recurrence escalation stops the cycle of temporary fixes.

Safe default (used automatically if it fails)

score 50, observation_count 0, no escalation.

Evaluation (technical)

- MAE of score vs the synthetic citizen's hidden honesty (Beta vs point counter)
- False-positive rate on new honest citizens
- Escalation precision and recall on real recurring locations

Business outcome

- Average days to a permanent fix after escalation
- Backlog reduction
- Citizens recognised (opt-in)

C08 · External Factors

Owner	Technique group	Acceptance tests
Mohammad	Cognitive Systems + Decision Automation	services/api/tests/acceptance/test_c08_external.py

Purpose

Adds context the request itself does not contain: how close it is (by road) to hospitals, schools, senior centres and transit, plus the current weather stress, so urgency and priority can reason about impact.

Inputs (locked)

RequestFacts

Field	Type	Required	Meaning
request_id	str	yes	
created_at	datetime	yes	
borough	Borough	yes	
request_type_code	str	yes	
complaint_type	str	yes	
descriptor	str?	no	
complaint_family	str	yes	
location_type	str?	no	
incident_address	str?	no	
incident_zip	str?	no	
address_type	str?	no	
bbl	str?	no	
latitude	float?	no	
longitude	float?	no	
channel	str	yes	
citizen_id	str?	no	
free_text	str?	no	

Outputs (locked)

ExternalFactorsResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
facilities	list[NearbyFacility]	no	
near_hospital	bool	no	
near_school	bool	no	
near_senior_center	bool	no	
on_major_road	bool	no	
context_score	float	no	
snapped_node_id	int?	no	
snap_distance_m	float?	no	
weather_severity	float	no	Current weather stress in this borough
reasons	list[Reason]	no	

How it works inside

1. Per-borough OSM drive graph and POIs cached locally (osmnx).
2. Snap the request to the nearest road node.
3. Straight-line vs network distance to the nearest facilities.
4. Threshold rules turn distances into explainable flags (near_hospital, and so on) and a context score.

Why it works and why it is better

A pothole outside a hospital entrance matters more than one on a quiet street; road distance reflects real access better than a straight line.

Safe default (used automatically if it fails)

No facilities, flags False, context_score 0 (neutral).

Evaluation (technical)

- Enrichment coverage
- Snapping distance distribution
- Agreement on hand-checked near/far cases
- Straight-line vs network disagreement

Business outcome

- Share of requests with context attached
- Sensitive-site requests served within target

C09 · Resolution Time Prediction

Owner	Technique group	Acceptance tests
Mohammad	Knowledge Discovery	services/api/tests/acceptance/test_c09_resolution.py

Purpose

Predicts how long the request will take, shown to the citizen as an honest range at stage 2 and used by prioritisation.

Inputs (locked)

RequestFacts

Field	Type	Required	Meaning
request_id	str	yes	
created_at	datetime	yes	
borough	Borough	yes	
request_type_code	str	yes	
complaint_type	str	yes	
descriptor	str?	no	
complaint_family	str	yes	
location_type	str?	no	
incident_address	str?	no	
incident_zip	str?	no	
address_type	str?	no	
bbl	str?	no	
latitude	float?	no	
longitude	float?	no	
channel	str	yes	
citizen_id	str?	no	
free_text	str?	no	

Outputs (locked)

ResolutionTimeResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
predicted_hours	float	yes	
p10_hours	float	yes	
p90_hours	float	yes	
eta_from	datetime	yes	
eta_to	datetime	yes	
top_features	list[Reason]	no	
reasons	list[Reason]	no	

How it works inside

1. Target: real closed minus created hours (resolution_valid rows only).

2. Features known at intake only: type, borough, time, holiday, weather, channel, backlog.
3. Baseline: median per type and borough. Model: gradient boosting or quantile regression giving p10 / p50 / p90.

Why it works and why it is better

A range calibrated on real history builds trust; a single number would be wrong most of the time.

Safe default (used automatically if it fails)

Family-level default range, status DEGRADED.

Evaluation (technical)

- MAE and RMSE on log-hours, R^2
- Interval coverage (share of true times inside p10 to p90)
- MAE by agency

Business outcome

- Citizen follow-up calls ('where is my request?')
- Satisfaction with the ETA

C10 · Urgency Assignment

Owner	Technique group	Acceptance tests
Khalid	Decision Automation	services/api/tests/acceptance/test_c10_urgency.py

Purpose

Assigns an explainable urgency level (CRITICAL, HIGH, MEDIUM, LOW) from the hazard, the location context and the time sensitivity.

Inputs (locked)

RequestFacts

Field	Type	Required	Meaning
request_id	str	yes	
created_at	datetime	yes	
borough	Borough	yes	
request_type_code	str	yes	
complaint_type	str	yes	
descriptor	str?	no	
complaint_family	str	yes	
location_type	str?	no	
incident_address	str?	no	
incident_zip	str?	no	
address_type	str?	no	
bbl	str?	no	
latitude	float?	no	
longitude	float?	no	
channel	str	yes	
citizen_id	str?	no	
free_text	str?	no	

DuplicateResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
relation	DuplicateRelation	yes	
duplicate_group_id	str	yes	
is_primary	bool	yes	
duplicate_of	str?	no	
recurrence_of	str?	no	
match_confidence	float	yes	
corroboration_count	int	no	Distinct reporters in the group
distinct_channel_count	int	no	
auto_close_recommended	bool	no	
match_evidence	list[Reason]	no	
reasons	list[Reason]	no	

ExternalFactorsResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
facilities	list[NearbyFacility]	no	
near_hospital	bool	no	
near_school	bool	no	
near_senior_center	bool	no	
on_major_road	bool	no	
context_score	float	no	
snapped_node_id	int?	no	
snap_distance_m	float?	no	
weather_severity	float	no	Current weather stress in this borough
reasons	list[Reason]	no	

CredibilityResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
credibility_score	float	yes	
observation_count	int	yes	
credibility_band	str	yes	
escalation_flag	bool	no	
escalation_level	EscalationLevel	no	
recurrence_count	int	no	Closures at this place and type in the window
evidence_trail	list[str]	no	request_ids proving the pattern
reasons	list[Reason]	no	

FakeRiskResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
unfounded_probability	float?	no	
risk_band	RiskBand?	no	
recommended_action	FakeAction	no	
reporter_burst_flag	bool	no	
contributing_factors	list[Reason]	no	
reasons	list[Reason]	no	

ResolutionTimeResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
predicted_hours	float	yes	
p10_hours	float	yes	
p90_hours	float	yes	
eta_from	datetime	yes	
eta_to	datetime	yes	
top_features	list[Reason]	no	

reasons	list[Reason]	no	

Outputs (locked)

UrgencyResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
urgency_level	UrgencyLevel	yes	
urgency_score	float	yes	
urgency_confidence	float	yes	
manual_review_flag	bool	no	
urgency_reasons	list[Reason]	no	

How it works inside

1. Forward-chaining IF-THEN rules over facts: type default, hazard words, sensitive site, storm, corroboration, escalation.
2. A weighted score orders requests within a level.
3. Safety rule: low credibility or fake risk may flag review but never lowers a hazard.
4. Every fired rule is written to urgency_reasons.

Why it works and why it is better

Rules are auditable and safe for rare critical cases where labelled data is scarce; the benchmark against learned models shows the trade-off.

Safe default (used automatically if it fails)

Catalog default urgency, manual_review_flag True.

Evaluation (technical)

- Critical/High recall (main safety metric)
- Critical false-negative rate
- Weighted Cohen's kappa vs the rubric labels
- Slices by borough and type

Business outcome

- Critical/High acknowledged within target
- Urgent cases downgraded or upgraded after review

C11 · Request Prioritization

Owner	Technique group	Acceptance tests
Khalid	Decision Automation + Resource Optimization	services/api/tests/acceptance/test_c11_priority.py

Purpose

Turns everything above into one ranked, explainable queue, and is the single place where the safety guardrails live.

Inputs (locked)

A queue of QueueItem (RequestFacts + all enrichments + due_at + previous_rank) for every open primary request.

Outputs (locked)

PriorityResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
priority_score	float	yes	
priority_rank	int	yes	
priority_band	PriorityBand	yes	
ranking_confidence	float	yes	
manual_review_flag	bool	no	
rank_timestamp	datetime	yes	
rank_change	int	no	
guardrails_applied	list[str]	no	e.g. ['G1_CRITICAL_FIRST']
priority_reasons	list[Reason]	no	

How it works inside

1. Weighted multi-criteria score: urgency, waiting time, SLA risk, corroboration, escalation, context, effort.
2. Guardrails: G1 CRITICAL first; G2 trust and fake risk scale the score by at most 0.6, never to 0; G3 failed components cannot lower a score; G4 overdue gets a floor.
3. Bands ACT_NOW / NEXT / PLANNED / MONITOR, rank change and review flags.
4. Extension: GA vs simulated annealing under crew capacity.

Why it works and why it is better

Weights make trade-offs visible and tunable; hard guardrails make it safe regardless of any single model.

Safe default (used automatically if it fails)

Urgency level, then first-in-first-out (also the FIFO baseline).

Evaluation (technical)

- Precision/recall@K, NDCG@K, Kendall tau vs the expert rank
- SLA violations, starvation, rank stability

Business outcome

- Urgent requests done within target

- Overdue backlog
- Waiting-time gap between boroughs (fairness)

C12 · Crew Deployment & Route Optimization

Owner	Technique group	Acceptance tests
Mohammad	Resource Optimization	services/api/tests/acceptance/test_c12_dispatch.py

Purpose

Assigns qualified crews to the ranked requests and orders their visits, trading off travel against priority.

Inputs (locked)

DispatchPlanInput

Field	Type	Required	Meaning
plan_id	str	yes	
as_of	datetime	yes	
horizon_minutes	int	no	
borough	Borough?	no	None = plan all boroughs independently
candidates	list[DispatchCandidate]	yes	
crews	list[CrewState]	yes	
algorithm	str	no	
random_seed	int	no	

Outputs (locked)

DispatchPlan

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
plan_id	str	yes	
as_of	datetime	yes	
algorithm	str	yes	
routes	list[CrewRoute]	yes	
unserved	list[UnservedRequest]	no	
metrics	DispatchMetrics	yes	
reasons	list[Reason]	no	

How it works inside

1. A* on the OSM road graph (vs Dijkstra) builds the travel-time matrix.
2. Baseline: nearest eligible crew / FIFO.
3. Genetic algorithm: chromosome = order + crew grouping; fitness = km + priority-weighted wait + constraint penalties.
4. Hard constraints: skill, agency, shift, capacity. Unserved requests get reason codes.

Why it works and why it is better

Greedy assignment wastes kilometres and delays urgent jobs; the GA searches the combined problem while staying explainable through the metrics.

Safe default (used automatically if it fails)

An empty plan (the admin assigns by hand); later the nearest-crew baseline.

Evaluation (technical)

- Total km vs baseline, optimality gap
- Priority-weighted waiting
- % priority requests served
- Constraint violations = 0
- Runtime

Business outcome

- % reduction in crew travel
- Jobs per crew-shift

C13 · Advanced Predictive Alerting

Owner	Technique group	Acceptance tests
Sankalp	Knowledge Discovery	notebook evaluation

Purpose

Warns the admin which kinds of requests are about to surge in which borough (for example drains in heavy rain), before citizens report them.

Inputs (locked)

DemandForecastInput

Field	Type	Required	Meaning
as_of	datetime	yes	
borough	Borough	yes	
horizon_hours	int	no	
weather	WeatherSnapshot	yes	

Outputs (locked)

DemandForecastResult

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
borough	Borough	yes	
alerts	list[DemandAlert]	no	

How it works inside

1. Training table: daily counts per borough x family from 2020 onward joined with Open-Meteo weather.
2. Baseline: normal-weather mean by weekday plus threshold lift rules. Model: Poisson GLM vs gradient boosting on counts.
3. An alert fires when expected / baseline and the spike probability pass thresholds; drivers are listed.

Why it works and why it is better

Moving from reactive to proactive: crews can be pre-positioned, and the forecast is checked against what citizens actually file.

Safe default (used automatically if it fails)

No alerts, status shown as degraded on the dashboard.

Evaluation (technical)

- MAE and Poisson deviance of daily counts
- Precision and recall of spike alerts on held-out storms

Business outcome

- % of predicted demand that matches requests filed
- Response time on storm days

C14 · Admin Natural-Language Q&A

Owner	Technique group	Acceptance tests
To be agreed	Cognitive Systems	services/api/tests/acceptance/test_c14_admin_qa.py

Purpose

Lets municipality staff ask plain questions ('How many high urgency drain requests are open in Brooklyn?') instead of building filters.

Inputs (locked)

AdminQuestion

Field	Type	Required	Meaning
question	str	yes	e.g. How many high urgency drain requests are open in Queens?
admin_id	str	yes	

Outputs (locked)

AdminAnswer

Field	Type	Required	Meaning
meta	ComponentMeta	yes	component id, version, model version, status OK/DEGRADED/FAILED, latency
matched_intent	str	yes	e.g. COUNT_OPEN_BY_FILTER
filters	dict[str, Any]	no	
answer_text	str	yes	
table	list[dict[str, Any]]	no	
confidence	float	yes	
suggestions	list[str]	no	Shown when confidence is low

How it works inside

1. Template intents (count, top types, averages, trends, why ranked) + slot extraction.
2. Answers only from read-only rows passed in; no SQL generation.
3. Low confidence leads to suggested questions.

Why it works and why it is better

Committed in the proposal's Cognitive Systems scope; templates keep it safe and accurate.

Safe default (used automatically if it fails)

UNKNOWN intent with example questions.

Evaluation (technical)

- Intent accuracy, slot F1, answer correctness on about 60 hand-written questions

Business outcome

- Time for a manager to answer an operational question

Appendix: status state machine and enums

From	Allowed next statuses
SUBMITTED	CANCELLED, DUPLICATE_LINKED, PROCESSED, REFERRED
PROCESSED	CANCELLED, DISPATCHED, DUPLICATE_LINKED, INVALID, REFERRED, SUBSIDED
DUPLICATE_LINKED	INVALID, PROCESSED, RESOLVED, SUBSIDED
DISPATCHED	ON_SITE, PROCESSED
ON_SITE	INVALID, PROCESSED, REFERRED, RESOLVED, SUBSIDED
RESOLVED	(terminal)
INVALID	(terminal)
SUBSIDED	(terminal)
REFERRED	(terminal)
CANCELLED	(terminal)

Borough: MANHATTAN, BROOKLYN, QUEENS, BRONX, STATEN ISLAND

AgencyCode: DOT, DEP, DSNY, DPR, HPD, DOHMH, NYPD, DOB, OTHER

CrewSkill: ROAD_REPAIR, STREET_LIGHTING, TRAFFIC_SIGNALS, SIDEWALK_REPAIR, SEWER_DRAINAGE, WATER_MAINS, SANITATION_COLLECTION, STREET_CLEANING, VEHICLE_REMOVAL, SNOW_ICE, FORESTRY, HOUSING_INSPECTION, PEST_CONTROL, NOISE_PATROL, PARKING_ENFORCEMENT, BUILDING_INSPECTION

UrgencyLevel: CRITICAL, HIGH, MEDIUM, LOW

PriorityBand: ACT_NOW, NEXT, PLANNED, MONITOR

CrewOutcomeCode: VALID_RESOLVED, VALID_NEEDS_FOLLOWUP, INVALID_NOT_FOUND, SUBSIDED, WRONG_AGENCY, NO_ACCESS